# Database Systems Implementation
# Mini-Project

**1075201**

*Computer Science (Part C)*

Hilary 2026

In all the pseudocode, we avoid showing logging and error handling details for clarity, this can be referenced in the codebase.

# 1

## a

### i

We manage tuple-level locks using shared/exclusive locks (`XSLocks` in Figure 1). These are standard two-phase locks: they operate in shared mode for queries and exclusive mode for edits (add/delete).

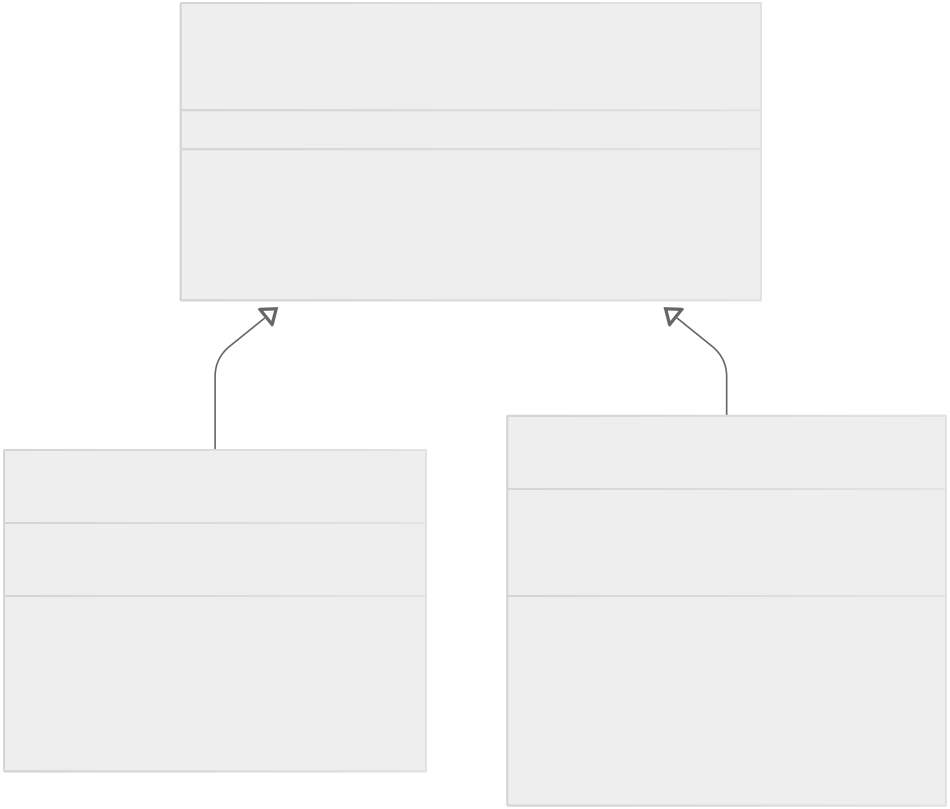

Figure 1: Lock Hierarchy

XSLocks are directly incorporated into the `DataTuple` class (Figure 2). The lock maintains an explicit set of `lock_holders` (`std::flat_set`) to support the deadlock detector's traversal of the waits-for graph. We use a flat set because the number of concurrent holders for a single tuple is typically very small, which improves cache locality.



Figure 2: DataTuple Class

Within each `Lock`, we maintain an explicit set of transactions holding the lock. This is later used by the deadlock detector to determine which transactions are waiting on which locks while traversing the implicit waits-for graph. We explicitly use inheritance and a VTable to implement the lock hierarchy, rather than a C++20 concept. This allows each transaction to hold sets of `Lock` pointers and treat them uniformly.

**ii**

To support serializable isolation and prevent phantoms, we implement predicate locks using shared locks (`SLocks` in Figure 1). These locks represent conditions of the form "all tuples matching predicate $P$".



Figure 3: Relation and Group Classes

We provide three levels of granularity for predicate locks:

1. Relation Lock: Covers all tuples in a relation $R(x, y)$.
2. Group Locks: Cover subsets of a relation based on indices. These handle conditions of the form $R(c, x)$, $R(x, c)$, and $R(x, x)$ (diagonal).
3. Tuple Locks: These act as the finest grain of locking, locking individual tuples.

Relation locks belong directly to `Relation` objects (see Figure 3), while group locks are stored in `Groups` belonging to relations (see Figure 3). The group locks are organized into three separate groups: one for the left attribute, one for the right attribute, and one for the diagonal (where left == right). We will later see in 1(c) how these granularities of locks neatly cover the types of queries we have to evaluate.

The different operations interact with the locks as follows:

Queries read ranges of tuples matching a predicate. For example, a query $R(1, Y)$ requires reading all tuples where the left attribute is 1 in relation $R$. To prevent phantoms, we acquire a shared lock on the relevant group lock (e.g., the group lock for left attribute 1). This ensures that if another transaction tries to insert a tuple that matches this predicate, it is blocked until the query transaction commits or rolls back. While reading existing tuples, we also need to prevent dirty reads from transactions that have not yet committed (or have aborted). Thus, queries call `Transaction::get_read_permit`, which checks `XSLock::permits_read` and records the tuple lock in `required_locks` when permission is denied.

There is an edge case for queries of the form $R(c_1, c_2)$, i.e. constant queries. For these queries, we do not take any predicate lock covering the $(c_1, c_2)$ tuple in $R$, and thus must acquire the tuple-level lock rather than only using `XSLock::permits_read`.

Edit operations modify tuples from an explicit list. As they do not use implicit ranges, they do not need to acquire predicate locks directly[1]. However, they still need to avoid causing phantoms, so for every

[1]Thus we do not use `XSLocks` for predicate locks.

tuple they edit, they must ensure that no other transaction has a shared lock on the relevant group lock covering that tuple's predicate. This is achieved using the `SLock::permits_edit` method, which checks whether the lock could be acquired in exclusive mode by this transaction. When editing a tuple, a transaction also acquires an exclusive lock on the tuple itself, as in standard two-phase locking.

Apart from the lock objects and their owners (`Group`, `Relation` and `DataTuple`), we also describe the rest of the lock management system, starting with `Transaction` objects (Figure 4) which hold onto pointers to locks.

Figure 4: Transaction Class

We track the locks held by each transaction in order to release the locks when the transaction commits or rolls back, and also track the locks that a transaction is relying on to make progress in order to detect deadlocks. A transaction likely holds many locks, so we use a hash set (`std::unordered_set`) to store the locks held by each transaction. On the other hand, we expect the number of locks that a transaction is waiting on to be small, so we use a `std::flat_set` to store the locks that a transaction is waiting on.

These transactions are stored in a single `Database` object (Figure 5) that acts as the global transaction and lock manager. When transactions finish an operation, they are either suspended or ready to

continue. The `Database` class is responsible for detecting deadlocks in the former case and progressing normal control flow in the latter case. This is implemented in `Database::OnControl`.

Figure 5: Database Class

We move some of the deadlock detection logic into a separate `DeadlockDetector` class (Figure 6), composed into the `Database` class. This keeps the core transaction/locking management logic separate from deadlock detection.

Figure 6: Deadlock Detector Class

**iii**

The locks expose the following primary functions:

`Lock::acquire(tx_id: TID, mode: LockMode) -> boolean`

Attempts to register the transaction as a holder. For XSLocks, this includes checking for shared/exclusive compatibility (see Algorithm 1). If a conflict occurs (e.g., requesting EXCLUSIVE when another transaction holds the lock), it returns `false`. For SLocks, we can always add the transaction as a holder and return `true`.

`Lock::release(tx_id: TID)`

Removes the transaction from the holder set. This is called during commit or rollback via

`Transaction::release_all_locks()`, which both releases locks and clears the transaction's `held_locks` set.

`Lock::current_holders() -> &Set<TID>`
Returns the transactions currently holding the lock, used by the deadlock detector to construct the waits-for graph.

`SLock::permits_edit(tx_id: TID) -> boolean`
Checks whether the transaction can acquire the lock in exclusive mode, used by edit operations to prevent phantoms. This returns `false` if there is another transaction holding the lock (apart from `tx_id` itself), and `true` otherwise.

`XSLock::permits_read(tx_id: TID) -> boolean`
Checks whether the transaction can acquire the lock in shared mode, used by query operations to prevent dirty reads. This returns `false` if there is another transaction holding the lock in exclusive mode (apart from `tx_id` itself), and `true` otherwise.

**Algorithm 1: XSLock::Acquire**

```
procedure XSLock::Acquire(tx_id: TID, mode: LockMode) → boolean
    if mode = EXCLUSIVE then
        if holder_count() >= 2 then
            return false
        else if holder_count() = 1 and ¬ is_held_by(tx_id) then
            return false
        else
            # Upgrade or Acquire
            lock_holders.insert(tx_id)
            held_exclusively ← true
            return true
        end if
    else
        if held_exclusively then
            return is_held_by(tx_id)
        else
            lock_holders.insert(tx_id)
            return true
        end if
    end if
end procedure
```

The transaction objects expose the following primary functions related to lock management:

`Transaction::acquire(lock: Lock, mode: LockMode) -> boolean`
A wrapper around `Lock::acquire` used by the execution engine. If `Lock::acquire` fails, the transaction is added to the lock's internal wait-queue and the transaction's `required_locks` set. The function returns `false` to signal that the transaction must be suspended.

`Transaction::get_read_permit(lock: XSLock) -> boolean`
Checks whether the transaction is allowed to read a tuple lock (`lock.permits_read(tid)`). If not,

the lock pointer is inserted into `required_locks` before returning `false`, so suspended query stages expose the correct waits-for dependencies.

The deadlock detector object provides the following function:

`DeadlockDetector::detect_cycle(tid: TID) -> Optional<TID>`
Performs a depth-first search starting from the given transaction ID, traversing the waits-for graph by following locks that the transaction is waiting on and the transactions currently holding those locks. If we encounter a back edge, we have found a cycle. We then locate the youngest transaction in the cycle and return it as the victim to roll back. This is described in more detail in Algorithm 2.

**Algorithm 2: Deadlock Detection**

```
procedure DeadlockDetector::DetectCycle(start_tid: TID) → Optional<TID>
    visited_tx.clear()
    cycle.clear()
    if DFS(start_tid) then
        # Locate the youngest transaction in the cycle to abort
        victim ← cycle[0]
        for each tid in cycle do
            if transactions[tid].born_at > transactions[victim].born_at then
                victim ← tid
            end if
        end for
        return victim
    end if
    return empty
end procedure
procedure DeadlockDetector::DFS(tid: TID) → boolean
    if visited_tx[tid] = VISITING then return true
    if visited_tx[tid] = VISITED then return false
    visited_tx[tid] ← VISITING
    for each lock in transactions[tid].required_locks do
        for each holding_tid in lock.current_holders() do
            if holding_tid = tid then continue
            if DFS(holding_tid) then
                cycle.push(tid)
                return true
            end if
        end for
    end for
    visited_tx[tid] ← VISITED
    return false
end procedure
```

## b

### i

The tuples themselves are stored in the `Relation` class's tuple container. We use a `StableVector` implemented as a linked list of fixed-size blocks. This allows us to keep stable pointers to tuples that can be stored in the various indices. Using a normal `std::vector` would require updating all pointers in the indices every time the vector resized, while using a plain linked list would lead to worse cache performance and allocation overhead. Note that we only ever append tuples to the StableVector and never remove tuples from the middle; deletion is represented by setting the `alive` flag to false in the DataTuple.

Based on the query patterns discussed in 1(c), we maintain three kinds of hash indices on each relation. The first two are for queries of the form $R(c, Y)$ and $R(X, c)$, where we want to efficiently iterate over all tuples matching a constant on the left or right attribute, respectively. The third index is for queries of the form $R(X, X)$, where we want to efficiently iterate over all tuples whose left and right attributes are equal. Since each of these indices covers the same range as the predicate locks described above, we also place them in the `Group` class.

Note that we do not maintain a whole-relation index. We chose not to do this because it would increase tuple insertion overhead without providing a significant speed-up: we can already use the left/right indices to efficiently check whether a tuple exists. To iterate over all tuples in a relation, we directly scan the tuples `StableVector`, which is more efficient than maintaining an additional index for this purpose.

### ii

We first discuss the methods exposed by the `Relation` class:

`Relation::get_tuple(left: uint32_t, right: uint32_t) -> DataTuple*`
retrieves a pointer to the `DataTuple` with the given left and right attributes. It creates the tuple if necessary, so it never returns `null`. Note that it does not guarantee that the tuple is alive, so the caller should check the `alive` flag if they only want alive tuples. This works by using `l_to_r_index` to find the group of tuples with the given left attribute, and then using `Group::find` to find the tuple with the given right attribute within that group.

`Relation::edit_tuple(left: uint32_t, right: uint32_t) -> boolean`
is used to add or delete a tuple from the relation by setting its `alive` flag. It first finds the `DataTuple` via `Relation::get_tuple`, then checks the relevant locks to prevent phantoms and tries to acquire an exclusive lock on the tuple. If any lock check or acquisition fails, it records the locks the transaction needs and returns `false` to signal that the transaction should be suspended. Otherwise, it updates the tuple's alive status and returns `true`.

Then we discuss the methods exposed by the `Group` class:

`Group::insert(tp: *DataTuple): unit` inserts the given tuple pointer into the group index. It is used when adding a new tuple to the relation and updating the indices.

`Group::find(left: uint32_t, right: uint32_t) -> DataTuple*` finds the tuple with the given left and right attributes in the group index. It returns `null` if no such tuple exists. It is used to efficiently locate tuples matching a given predicate when evaluating queries.

## c

To best deal with suspensions, we choose to use a volcano style query evaluation algorithm which naturally handles suspensions in between calls of `next()`. This comes in 2 phases: setting up the query pipeline, and pulling tuples through the pipeline.

The query pipeline for a query $Q = R_1(l_1, r_1), ..., R_n(l_n, r_n)$ consists of $n + 1$ `Stages`. Stage 0 is always an initial stage while stage $i$ for $i > 0$ corresponds to the effect of the query atom $R_i(l_i, r_i)$.



Figure 7: Stage Class

Each `Stage` (apart from the initial stage) is connected to a previous `Stage` via the `previous` pointer, when `next()` is called on a stage, it can pull tuples from the previous stage via `previous.next()` as many times as is needed to produce the next output tuple. While stages conceptually pass tuples to each other, in practise, no stage ever modifies the tuples it receives from the variables set in tuples by previous stages, thus they all directly share the same buffer to write to: `channel`.

The building of the pipeline is initiated by `Transaction::StartQuery` when a query is first executed. This function handles constructing the stages and connecting them together. All the stage specific configuration is handled by the `Stage` constructor. This differentiates between the different types of stages by considering the structure of the query Atom with respect to the previously seen variables and also acquires the relevant predicate locks.

**Algorithm 3: Transaction::StartQuery**

```
procedure Transaction::StartQuery(query: Vec<QueryAtom>)
            → StatusCode
  state ← EXECUTING_QUERY
  num_answers ← 0
  # Gather build variable to index mapping
  num_vars ← 0
  var_idx.clear()
  for each atom in query do
    for each arg in {atom.left, atom.right} do
      if arg is Variable then
        if arg.name ∉ var_idx then
          var_idx[arg.name] ← num_vars
          num_vars ← num_vars + 1
        end if
      end if
    end for
  end for
  # Build query pipeline stages
  this.query_atoms ← query
  channel.resize(num_vars)
  stages.clear()
  for i ← 0 to query_atoms.len() do
    stages.push(Stage(i, *this))
  end for
  return resume_query()
end procedure
```

**Algorithm 4: Stage::Stage**

```
procedure Stage::Stage(stage_idx: usize, tx: &Transaction)
  if stage_idx = 0 then
    type ← INITIAL
    return
  end if
  atom ← tx.query_atoms[stage_idx - 1]
  previous ← tx.stages[stage_idx - 1]
  rel ← tx.relations[atom.relation]
  num_input_vars ← previous.num_output_vars
  match (atom.left, atom.right)
    (Constant(c1), Constant(c2)) ⇒
      type ← CONST_CONST
      num_output_vars ← num_input_vars
    (Constant(c), Variable(v)) ⇒
```

```
        var_idx ← tx.var_idx[v.name]; const_val ← c.value
        group ← rel.l_to_r_index[const_val]; group_setup()
    (Variable(v), Constant(c)) ⇒
        var_idx ← tx.var_idx[v.name]; const_val ← c.value
        group ← rel.r_to_l_index[const_val]; group_setup()
    (Variable(v1), Variable(v2)) ⇒
        idx1 ← tx.var_idx[v1.name]; idx2 ← tx.var_idx[v2.name]
        var_idx ← idx1; var2_idx ← idx2
        if idx1 = idx2 then
            group ← rel.diagonal_index; group_setup()
        else
            if idx1 ≥ num_input_vars and idx2 ≥ num_input_vars then
                type ← RELATION_PRODUCT; num_output_vars ← num_input_vars + 2
                tx.acquire(rel.whole_rel_lock, SHARED)
            else if idx1 ≥ num_input_vars then
                type ← JOIN_RIGHT; num_output_vars ← num_input_vars + 1
            else if idx2 ≥ num_input_vars then
                type ← JOIN_LEFT; num_output_vars ← num_input_vars + 1
            else
                type ← RELATION_FILTER; num_output_vars ← num_input_vars
                tx.acquire(rel.whole_rel_lock, SHARED)
            end if
        end if
    end match
end procedure
procedure Stage::group_setup()
    tx.acquire(group.lock, SHARED)
    if var_idx ≥ num_input_vars then
        type ← GROUP_PRODUCT
        num_output_vars ← num_input_vars + 1
    else
        type ← GROUP_FILTER
        num_output_vars ← num_input_vars
    end if
end procedure
```

The second phase of the query algorithm involves repeatedly pulling tuples through the pipeline by calling `Stage::next()` from the last stage until it either returns `SUSPEND` or `FINISHED`. This is orchestrated by `Transaction::resume_query()`.

**Algorithm 5: Transaction::ResumeQuery**

```
procedure Transaction::ResumeQuery() → StatusCode
    while true do
```

```
    match stages.last().next()
      OK ⇒
        num_answers ← num_answers + 1
        print_tuple(query_channel)
        continue
      SUSPEND ⇒
        return StatusCode::SUSPENDED
      FINISHED ⇒
        state ← READY
        print "Number of answers: ", num_answers
        return StatusCode::SUCCESS
    end match
  end while
end procedure
```

`Stage::next()` matches over the stage type and calls one of the specific next functions that implements the following logic to produce the set of tuples $W'$ from the input set of tuples $W$ received from the previous stage:

- `next_const_const()`
  - $W' = W$ if $R(c_1, c_2)$ and $\emptyset$ otherwise
- `next_group_filter()`
  - For $R(x, c)$: $W' = \{t \in W \mid (t.x, c) \in R\}$
  - For $R(c, x)$: $W' = \{t \in W \mid (c, t.x) \in R\}$
  - For $R(x, x)$: $W' = \{t \in W \mid (t.x, t.x) \in R\}$
- `next_group_product()`
  - For $R(x, c)$: $W' = \{\langle t, x\rangle \mid t \in W, (x, c) \in R\}$
  - For $R(c, x)$: $W' = \{\langle t, x\rangle \mid t \in W, (c, x) \in R\}$
  - For $R(x, x)$: $W' = \{\langle t, x\rangle \mid t \in W, (x, x) \in R\}$
- `next_relation_filter()`
  - $W' = \{t \in W \mid (t.x_1, t.x_2) \in R\}$
- `next_relation_product()`
  - $W' = \{\langle t, x_1, x_2\rangle \mid t \in W, (x_1, x_2) \in R\}$
- `next_join_left()`
  - $W' = \{\langle t, x_2\rangle \mid t \in W, (t.x_1, x_2) \in R\}$
- `next_join_right()`
  - $W' = \{\langle t, x_1\rangle \mid t \in W, (x_1, t.x_2) \in R\}$

These are implemented in volcano style in a mostly straightforward way:

- `next_const_const` checks whether the constant tuple is in $R$ and forwards all or none of the tuples from the previous stage accordingly.
- `next_group_filter` and `next_relation_filter` call `previous.next()` and check whether the relevant part of the incoming tuple passes the filter criteria. If so we forward the tuple, otherwise we discard it and call `previous.next()` again until they find a tuple that passes or the previous stage is finished.
- `next_group_product` and `next_relation_product` maintain an iterator over the group or relation respectively to construct the cartesian product. With the same input tuple, they produce many output

tuples until the iterator is exhausted, then they call `previous.next()` to get a new input tuple and reset the iterator.

- `next_join_left` and `next_join_right` are similar to the `next_group_product` but instead of iterating over the same group all the time, the group they iterate over depends on the current input tuple from the previous stage, as they need to find tuples that match on one attribute with the input tuple.

All these methods need to remember to check for read permission from the relevant tuple level locks. The join operations additionally need to acquire the relevant group locks when they move to a new group based on the input tuple from the previous stage.

If any call to `previous.next()` suspends or any lock check/acquisition fails, the method needs to suspend and return `SUSPEND` without advancing the state of the stage, so that when it is resumed, it can retry the same operation from the same point. The stages ensure the state is maintained by not advancing the iterators for products and joins and by using the `call_next` boolean for filters to indicate whether the next call should call `previous.next()` or not.

We show pseudocode for `Stage::next_relation_filter()`, `Stage::next_group_product()`, and `Stage::next_join_left()` as examples of how the different stage types are implemented. The other stage types are implemented similarly based on the logic described above.

**Algorithm 6: Query Evaluation Methods**

```
procedure Stage::NextJoinLeft() → PipelineStatus
  loop
    if ¬ group_iter_valid then
      match previous.next()
        OK ⇒
          group ← rel.l_to_r_index[channel[var_idx]]
          group_iter ← group.tuples.begin()
          tx.acquire(group.lock, SHARED) # never fails
          group_iter_valid ← true
        other ⇒ return other
      end match
    end if
    while group_iter ≠ group.tuples.end() do
      tp ← group_iter.second
      if ¬ tx.get_read_permit(tp.lock) then return SUSPEND
      group_iter ← group_iter + 1
      if tp.alive then
        channel[var2_idx] ← tp.right
        return OK
      end if
    end while
    group_iter_valid ← false
  end loop
end procedure
procedure Stage::NextGroupProduct() → PipelineStatus
```

```
    loop
        if num_input_vars = 0 then
            # expected to be in group.tuples.begin() when set up
            # just 1 output tuple per group tuple
            match previous.next()
                OK ⇒
                    if group_iter = group.tuples.end() then return FINISHED
                other ⇒ return other
            end match
        else if group_iter = group.tuples.end() then
            # expected to be in group.tuples.end() when set up
            match previous.next()
                OK ⇒ group_iter ← group.tuples.begin()
                other ⇒ return other
            end match
        end if
        tp ← group_iter.second
        if ¬ tx.get_read_permit(tp.lock) then return SUSPEND
        group_iter ← group_iter + 1
        if tp.alive then
            if left_is_const then channel[var_idx] ← tp.right
            else channel[var_idx] ← tp.left
            return OK
        end if
    end loop
end procedure
procedure Stage::NextRelationFilter() → PipelineStatus
    loop
        if call_next then
            match previous.next()
                OK ⇒ pass
                other ⇒ return other
            end match
        end if
        tp ← rel.get_tuple(channel[var_idx], channel[var2_idx])
        if ¬ tx.get_read_permit(tp.lock) then
            call_next ← false
            return SUSPEND
        end if
        call_next ← true
        if tp.alive then return OK
    end loop
end procedure
```

We note that an alternative design for the stages would be to use inheritance and have a separate class for each stage type. This would reduce the number of state variables that are used to suspend the stages. However, as different types of stages share different subsets of the state variables, we felt it was unnecessarily complex to break down the stages into separate classes. Furthermore, doing so would require the different stages to be allocated separately rather than being stored in a contiguous vector, which would lead to worse cache performance and more complex memory management.

An alternatively algorithm all together would have been to use an eager set-at-a-time style algorithm for query evaluation. However, we felt that this would have worse memory performance due to the need to materialize a potentially large number of intermediate tuples at once, and is less natural to implement with suspensions.

## d & e

We use a single algorithm for both adding and deleting, with only a difference in the `new_alive` parameter. An important design consideration is that when a transaction fails either a lock permission check or a lock acquisition, all relevant locks are recorded in the transaction's `required_locks`. This helps ensure deadlocks are detected as early as possible (without needing to resume transactions that cannot make meaningful progress).

We also store the original tuple values to allow rollbacks.

**Algorithm 7: Single Tuple Edit Algorithm**

```
procedure Relation::edit_tuple(tx: &Transaction, left: uint32_t, right: uint32_t,
            new_alive: bool) → StatusCode
  tp ← get_tuple(left, right) # creates the tuple if it doesn't exist
  if permitted_by_locks(tx.tid, left, right) and tx.acquire(tp->lock, EXCLUSIVE) then
    if tp->alive ≠ new_alive then
      tx.store_original(tp)
      tx.num_modified++
      tp->alive ← new_alive
    endif
    return true;
  else
    insert_required_locks_for_edit(tx, left, right)
    return false;
  end if
end procedure
procedure Relation::permitted_by_locks(tx_id: TID, left: uint32_t, right: uint32_t)
            → boolean
  if ¬ whole_rel_lock.permits_edit(tx_id) then return false
  if left = right and ¬ diagonal_index.lock.permits(tx_id) then return false
  if ¬ l_to_r_index[left].lock.permits(tx_id) then return false
  if ¬ r_to_l_index[right].lock.permits(tx_id) then return false
  return true
end procedure
```

```
procedure Relation::insert_required_locks_for_edit(tx: &Transaction, left: uint32_t,
right: uint32_t) → unit
  # tp.lock already inserted via tx.acquire()
  tx.required_locks.insert(&whole_rel_lock)
  if left = right then
    tx.required_locks.insert(&diagonal_index.lock)
  end if
  tx.required_locks.insert(&l_to_r_index[left].lock)
  tx.required_locks.insert(&r_to_l_index[right].lock)
end procedure
```

## f

When we commit, we ensure that the transaction is in the "ready" state and then release all locks.

**Algorithm 8: Commit Algorithm**

```
procedure Transaction::commit() → StatusCode
  if state ≠ READY then
    print("Error: Trying to commit a transaction that is not ready.")
    return StatusCode::SUCCESS;
  end if
  for lock in held_locks do
    lock.release(tid)
  end for
  held_locks.clear()
  return StatusCode::SUCCESS;
end procedure
```

## g

To roll back, we restore the original tuple values and release all held locks.

**Algorithm 9: Rollback Algorithm**

```
procedure Transaction::rollback() → StatusCode
  for (tp, original_alive) in original_values do
    tp->alive ← original_alive
  end for
  for lock in held_locks do
    lock.release(tid)
  end for
  held_locks.clear()
  return StatusCode::SUCCESS;
end procedure
```

# 3

## a

| | Action | Time (ms) | # of Answers |
|---|---|---|---|
| Step 1 | Import: | 23308 | |
| | Rollback: | 5145 | |
| Step 2 | Import: | 22480 | |
| | Commit: | 3363 | |
| Step 3 | Query 1: | 0 | 4 |
| | Query 2: | 1735 | 264 |
| | Query 3: | 0 | 6 |
| | Query 4: | 0 | 34 |
| | Query 5: | 0 | 719 |
| | Query 6: | 224 | 1048532 |
| | Query 7: | 0 | 67 |
| | Query 8: | 8 | 7790 |
| | Query 9: | 1150 | 27247 |
| | Query 10: | 0 | 4 |
| | Query 11: | 0 | 224 |
| | Query 12: | 0 | 15 |
| | Query 13: | 0 | 472 |
| | Query 14: | 112 | 795970 |
| | Commit: | 42 | |
| Step 4 | Delete: | 811 | |
| | Rollback: | 247 | |
| Step 5 | Delete: | 3509 | |
| | Commit: | 358 | |
| Step 6 | Query 1: | 1 | 0 |
| | Query 2: | 139 | 0 |
| | Query 3: | 0 | 0 |
| | Query 4: | 0 | 0 |
| | Query 5: | 0 | 0 |
| | Query 6: | 198 | 923871 |
| | Query 7: | 0 | 0 |
| | Query 8: | 13 | 4040 |
| | Query 9: | 846 | 22522 |
| | Query 10: | 0 | 0 |
| | Query 11: | 0 | 186 |
| | Query 12: | 0 | 9 |
| | Query 13: | 2 | 413 |
| | Query 14: | 103 | 701364 |

| Action | Time (ms) | # of Answers |
|---|---|---|
| Commit: | 25 | |

Table 1: Benchmarking Results

## b

The results show that my database implementation is more optimised for query-heavy workloads than for write-heavy workloads. This makes sense since editing requires acquiring a larger number of locks, populating sets such as each transaction's `held_locks`, and adding tuples to the various group indices. By contrast, queries typically need fewer lock acquisitions because predicate locks can cover multiple tuples, and these locks are SLocks, which are always acquirable in shared mode.

It is possible to trade off between these behaviours by increasing or decreasing the number of indices. For example, we could have added an extra index over the whole relation, or we could have created group locks without group indices. However, I believe my choice is fairly sensible and avoids any obviously bad query pattern.

In terms of scalability with respect to the number of open transactions, performance is good when contention is low. If transactions mostly touch different groups/tuples, they can acquire disjoint locks and run largely independently, so increasing the number of open transactions does not hurt throughput much.

The main scalability bottleneck appears once contention rises. More open transactions means larger lock-holder sets and more required-lock edges, and each suspension triggers a DFS deadlock check over that waits-for graph (cost proportional to $V + E$). In addition, coarse predicates such as $R(x, y)$ take relation-level shared locks, which can block many edits at once. So the system scales well in low-conflict workloads, but degrades under high-conflict workloads because more time is spent suspended or in deadlock handling rather than doing useful work; this is the expected trade-off for serializable isolation.